

The important point is: **your code cannot be written exactly as shown directly inside a Java class body**. Java allows some declarations at class level, but a lambda expression assigned to a variable is **not a valid standalone class-level statement** unless it is part of a field declaration/initializer.

Let's break it down carefully.

1. Your original code

```
class Employee_Get_Salary_Lambda {  
  
    String name;  
    double salary;  
  
    public Employee_Get_Salary_Lambda(String name, double salary) {  
        super();  
        this.name = name;  
        this.salary = salary;  
    }  
}  
  
Function<Employee_Get_Salary_Lambda, Double> getSal = emp -> emp.salary;  
  
Employee_Get_Salary_Lambda empGetSalary =  
    new Employee_Get_Salary_Lambda("Anurag", 50000.0);  
  
System.out.println(getSal.apply(empGetSalary));
```

If you put all of this directly in a .java file outside a method, constructor, initializer, or field declaration, it **will not compile**.

2. Why doesn't it work outside a method?

Java has a strict class structure.

At the class level, you can have things such as:

```
class Test {  
  
    // fields  
    int x;  
  
    // constructors  
    Test() {  
    }  
  
    // methods
```

```
void test() {  
}  
  
// nested classes/interfaces  
class Inner {  
}  
}
```

But this:

```
System.out.println(...);
```

is an **executable statement**.

Executable statements cannot simply appear directly in the class body.

For example, this is invalid:

```
class Test {  
  
    int x = 10;  
  
    System.out.println(x); // ❌invalid  
}
```

You would get a compilation error.

3. But fields can have initialization expressions

This is where your lambda example becomes interesting.

This:

```
Function<Employee_Get_Salary_Lambda, Double> getSal =  
    emp -> emp.salary;
```

can actually be made a **field declaration**.

For example:

```
import java.util.function.Function;  
  
class Employee_Get_Salary_Lambda {
```

```

String name;
double salary;

public Employee_Get_Salary_Lambda(String name, double salary) {
    super();
    this.name = name;
    this.salary = salary;
}

static Function<Employee_Get_Salary_Lambda, Double> getSal =
    emp -> emp.salary;
}

```

This is valid.

Why?

Because:

```

Function<Employee_Get_Salary_Lambda, Double> getSal =
    emp -> emp.salary;

```

is a **field declaration with an initializer**, not a standalone statement.

4. Can empGetSalary also be outside a method?

Yes, if you make it a field.

For example:

```

static Employee_Get_Salary_Lambda empGetSalary =
    new Employee_Get_Salary_Lambda("Anurag", 50000.0);

```

Now both variables are class fields.

Complete example:

```

import java.util.function.Function;

class Employee_Get_Salary_Lambda {

    String name;
    double salary;

    public Employee_Get_Salary_Lambda(String name, double salary) {
        super();
        this.name = name;
    }
}

```

```

        this.salary = salary;
    }

    static Function<Employee_Get_Salary_Lambda, Double> getSal =
        emp -> emp.salary;

    static Employee_Get_Salary_Lambda empGetSalary =
        new Employee_Get_Salary_Lambda("Anurag", 50000.0);
}

```

This is valid Java.

But we still have a problem with:

```
System.out.println(getSal.apply(empGetSalary));
```

That is an executable statement.

It cannot appear directly in the class body.

5. How can we execute it without a main() method?

There are several possibilities.

One important Java mechanism is an **initializer block**.

Static initializer block

You can write:

```

static {
    System.out.println(getSal.apply(empGetSalary));
}

```

Now the complete program can be:

```

import java.util.function.Function;

class Employee_Get_Salary_Lambda {

    String name;
    double salary;

    public Employee_Get_Salary_Lambda(String name, double salary) {
        super();
        this.name = name;
    }
}

```

```

        this.salary = salary;
    }

    static Function<Employee_Get_Salary_Lambda, Double> getSal =
        emp -> emp.salary;

    static Employee_Get_Salary_Lambda empGetSalary =
        new Employee_Get_Salary_Lambda("Anurag", 50000.0);

    static {
        System.out.println(getSal.apply(empGetSalary));
    }
}

```

However, there is another important issue.

If you try to execute this class with:

```
java Employee_Get_Salary_Lambda
```

modern Java launch behavior generally expects an entry point. A static initializer by itself is **not a replacement for a main() method** for normal application launching.

So if your question is specifically:

"Can I execute this code without writing main()?"

the answer depends on **how the class is being loaded/executed**.

6. The easiest traditional solution: main()

The conventional solution is:

```

import java.util.function.Function;

public class Employee_Get_Salary_Lambda {

    String name;
    double salary;

    public Employee_Get_Salary_Lambda(String name, double salary) {
        super();
        this.name = name;
        this.salary = salary;
    }

    public static void main(String[] args) {

```

```

Function<Employee_Get_Salary_Lambda, Double> getSal =
    emp -> emp.salary;

Employee_Get_Salary_Lambda empGetSalary =
    new Employee_Get_Salary_Lambda("Anurag", 50000.0);

System.out.println(getSal.apply(empGetSalary));
}
}

```

Output:

50000.0

7. What exactly is happening with the lambda?

This:

```

Function<Employee_Get_Salary_Lambda, Double> getSal =
    emp -> emp.salary;

```

means:

Create a function that accepts an `Employee_Get_Salary_Lambda` object and returns its salary as a `Double`.

`Function<T, R>` is conceptually:

`Function<T, R>`

where:

T = input type

R = return type

Therefore:

`Function<Employee_Get_Salary_Lambda, Double>`

means:

Input → `Employee_Get_Salary_Lambda`

Output → `Double`

The lambda:

emp -> emp.salary

is equivalent conceptually to:

```
Double getSalary(Employee_Get_Salary_Lambda emp) {  
    return emp.salary;  
}
```

You can visualize it as:

Employee object

|

v

```
+-----+  
| name = "Anurag" |  
| salary = 50000   |  
+-----+  
|  
| getSal.apply(emp)  
v  
50000.0
```

8. What does apply() do?

Function has the abstract method:

R apply(T t);

For your particular Function:

Function<Employee_Get_Salary_Lambda, Double>

the method effectively becomes:

Double apply(Employee_Get_Salary_Lambda emp);

Your lambda provides the implementation:

emp -> emp.salary

So:

getSal.apply(empGetSalary)

means:

Call the Function

↓
Pass empGetSalary
↓
emp becomes empGetSalary
↓
return emp.salary
↓
50000.0

9. Lambda equivalent using an anonymous class

Your lambda:

```
Function<Employee_Get_Salary_Lambda, Double> getSal =  
    emp -> emp.salary;
```

is conceptually equivalent to the older anonymous-class syntax:

```
Function<Employee_Get_Salary_Lambda, Double> getSal =  
    new Function<Employee_Get_Salary_Lambda, Double>() {  
  
        @Override  
        public Double apply(Employee_Get_Salary_Lambda emp) {  
            return emp.salary;  
        }  
    };
```

Then:

```
System.out.println(getSal.apply(empGetSalary));
```

produces:

50000.0

10. What if you want absolutely no main()?

You can put execution into a **static initializer**:

```
import java.util.function.Function;
```

```
public class Employee_Get_Salary_Lambda {
```

```
    String name;  
    double salary;
```

```
    public Employee_Get_Salary_Lambda(String name, double salary) {
```

```

        this.name = name;
        this.salary = salary;
    }

    static Function<Employee_Get_Salary_Lambda, Double> getSal =
        emp -> emp.salary;

    static Employee_Get_Salary_Lambda empGetSalary =
        new Employee_Get_Salary_Lambda("Anurag", 50000.0);

    static {
        System.out.println(getSal.apply(empGetSalary));
    }
}

```

The important distinction is:

```
static Function<...> getSal = ...;
```

✓ Field declaration

```
static Employee_Get_Salary_Lambda empGetSalary = ...;
```

✓ Field declaration

```
static {
    System.out.println(...);
}
```

✓ Static initializer block

But:

```
System.out.println(...);
```

✗ Cannot be directly placed in the class body.

11. Instance initializer is another possibility

You can also use an **instance initializer**:

```

{
    System.out.println(getSal.apply(empGetSalary));
}

```

For example:

```

import java.util.function.Function;

class Employee_Get_Salary_Lambda {

    String name;
    double salary;

    public Employee_Get_Salary_Lambda(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    Function<Employee_Get_Salary_Lambda, Double> getSal =
        emp -> emp.salary;

    Employee_Get_Salary_Lambda empGetSalary =
        new Employee_Get_Salary_Lambda("Anurag", 50000.0);

    {
        System.out.println(getSal.apply(empGetSalary));
    }
}

```

But there is a serious problem with this particular example: creating an instance requires the constructor, and the instance initializer participates in object construction. It is therefore not a useful way to launch this example as a standalone program.

12. Static initialization order is important

Consider:

```

class Test {

    static int x = 10;

    static {
        System.out.println(x);
    }
}

```

When the class is initialized, Java processes static field initializers and static initializer blocks **in textual order**.

For example:

```

class Test {

```

```
static int x = 10;

static {
    System.out.println(x);
}

static int y = 20;
}
```

The rough sequence is:

```
Load class
↓
Initialize x
↓
Run first static block
↓
Initialize y
```

Therefore:

10

is printed.

This matters for your lambda example.

13. Your example with explicit initialization order

You could write:

```
import java.util.function.Function;

class Employee_Get_Salary_Lambda {

    String name;
    double salary;

    public Employee_Get_Salary_Lambda(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    static Function<Employee_Get_Salary_Lambda, Double> getSal =
        emp -> emp.salary;

    static Employee_Get_Salary_Lambda empGetSalary =
```

```

        new Employee_Get_Salary_Lambda("Anurag", 50000.0);

    static {
        Double salary = getSal.apply(empGetSalary);

        System.out.println(salary);
    }
}

```

Conceptually:

```

Class initialization
    ↓
Create getSal Function
    ↓
Create empGetSalary object
    ↓
Execute static initializer
    ↓
getSal.apply(empGetSalary)
    ↓
lambda receives empGetSalary
    ↓
emp.salary
    ↓
50000.0

```

14. An even cleaner class-level version

You can eliminate the separate empGetSalary field and initialize the result directly:

```

import java.util.function.Function;

class Employee_Get_Salary_Lambda {

    String name;
    double salary;

    public Employee_Get_Salary_Lambda(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    static Function<Employee_Get_Salary_Lambda, Double> getSal =
        emp -> emp.salary;

    static {
        Employee_Get_Salary_Lambda emp =

```

```

        new Employee_Get_Salary_Lambda("Anurag", 50000.0);

    System.out.println(getSal.apply(emp));
}
}

```

Here:

```

Employee_Get_Salary_Lambda emp =
    new Employee_Get_Salary_Lambda("Anurag", 50000.0);

```

is a **local variable inside the static initializer block**.

15. Very important distinction

You should remember these three categories for Java interviews:

Code	Location	Valid?
int x = 10;	Class body	✓ Field
Function<...> f = emp -> emp.salary;	Class body	✓ Field
new Employee(...);	Class body as declaration	✓ If assigned to field
System.out.println(...);	Class body	✗
System.out.println(...);	Method	✓
System.out.println(...);	Static initializer {}	✓
System.out.println(...);	Instance initializer {}	✓

The key rule is:

Java class bodies contain declarations, while executable statements must normally be inside a method, constructor, or initializer block.

16. Why does Java allow the lambda field?

This sometimes confuses people.

You might think:

```
emp -> emp.salary
```

is executable code, so why can it appear here?

Because the complete statement:

```
Function<Employee_Get_Salary_Lambda, Double> getSal =  
    emp -> emp.salary;
```

is a **field declaration**.

The lambda is the **initializer expression** of that field.

Compare:

```
int x = 10;
```

Here:

```
int x  
↓  
field declaration
```

```
10  
↓  
initializer
```

Similarly:

```
Function<Employee_Get_Salary_Lambda, Double> getSal =  
    emp -> emp.salary;
```

means:

```
Function<...> getSal  
↓  
field declaration
```

```
emp -> emp.salary  
↓  
field initializer
```

17. Is the lambda executed immediately?

This is another important concept.

When you write:

```
Function<Employee_Get_Salary_Lambda, Double> getSal =  
    emp -> emp.salary;
```

you are primarily creating/configuring a Function object/functional behavior.

The salary calculation happens when you invoke:

```
getSal.apply(empGetSalary);
```

Think of it as:

Create function

↓

getSal

↓

[waiting]

↓

apply(employee)

↓

execute lambda

↓

return salary

This is why lambda expressions are often described as **behavior passed as a value**.

18. Method reference equivalent

Since you're studying Java 8 method references, this example also connects directly to that topic.

Your lambda:

```
emp -> emp.salary
```

accesses an **instance field**, not an instance method.

Therefore you cannot directly write:

```
emp::salary
```

because salary is a field, not a method.

If you had:

```
double getSalary() {  
    return salary;  
}
```

then you could write:

```
Function<Employee_Get_Salary_Lambda, Double> getSal =  
    Employee_Get_Salary_Lambda::getSalary;
```

Now the method reference means:

Employee object



getSalary()



Double

So:

```
emp -> emp.getSalary()
```

can become:

```
Employee_Get_Salary_Lambda::getSalary
```

But:

```
emp -> emp.salary
```

cannot become:

```
Employee_Get_Salary_Lambda::salary // ✗
```

because Java method references reference **methods or constructors**, not ordinary fields.

Final takeaway

Your original code contains three different kinds of constructs:

```
Function<Employee_Get_Salary_Lambda, Double> getSal =  
    emp -> emp.salary;
```

Field declaration with lambda initializer → can exist at class level.

```
Employee_Get_Salary_Lambda empGetSalary =  
    new Employee_Get_Salary_Lambda("Anurag", 50000.0);
```

Field declaration with object initializer → can exist at class level.

```
System.out.println(getSal.apply(empGetSalary));
```

Executable statement → cannot exist directly in the class body; it must be inside a **method, constructor, static initializer, or instance initializer**.

For normal Java applications, the clearest and most maintainable version remains:

```
public static void main(String[] args) {  
  
    Function<Employee_Get_Salary_Lambda, Double> getSal =  
        emp -> emp.salary;  
  
    Employee_Get_Salary_Lambda empGetSalary =  
        new Employee_Get_Salary_Lambda("Anurag", 50000.0);  
  
    System.out.println(getSal.apply(empGetSalary));  
}
```

Output:

50000.0

Static Initializer vs Instance Initializer in Java

Java provides two important initialization mechanisms:

1. **Static initialization** — performed when the class is initialized, generally once per class.
2. **Instance initialization** — performed every time a new object is created.

The most important rule is:

Static initialization happens once per class; instance initialization happens once per object.

The order in which **variables, initializer blocks, and constructors** execute is especially important in Java interviews.

1. What Is a Static Initializer?

A **static initializer** is a block of code declared with the static keyword:

```
static {  
    // initialization code  
}
```

It belongs to the **class**, not to individual objects.

It executes when the JVM **initializes the class**, normally before the class is first actively used.

Example

```
class Employee {
```

```
static {  
    System.out.println("Static initializer");  
}  
}
```

When Employee is initialized, the static block executes.

2. What Is an Instance Initializer?

An **instance initializer** is a block of code without static:

```
{  
    // initialization code  
}
```

It belongs to an **object instance**.

It executes every time an object is created, as part of the constructor invocation process.

Example

```
class Employee {  
  
    {  
        System.out.println("Instance initializer");  
    }  
}
```

Every time you execute:

```
new Employee();
```

the instance initializer executes.

3. Basic Difference

Feature	Static Initializer	Instance Initializer
Syntax	static { }	{ }
Belongs to	Class	Object
Executes	During class initialization	During object creation

Frequency	Normally once per class initialization	Once per object
Can directly access instance variables	✗ No	✓ Yes
Can directly call instance methods	✗ No	✓ Yes
Can directly access static variables	✓ Yes	✓ Yes
Can directly call static methods	✓ Yes	✓ Yes
Runs before constructor	N/A	✓ Yes
Runs before instance methods are called	Not specifically	Yes, for a newly constructed object
Typical purpose	Class-level setup	Object-level setup

4. The Most Important Initialization Sequence

For a simple class, think of Java initialization in this order:

```

Class Loading
↓
Default static initialization
↓
Static variables + static blocks
↓
Class initialization complete
↓
new Object()
↓
Default instance initialization
↓
Instance variables + instance blocks
↓
Constructor
↓
Object ready

```

There is an important distinction:

Methods themselves are not automatically executed as part of initialization.

Methods are loaded/available, but their bodies execute only when they are called.

5. Example: Static Variable, Static Block, Instance Variable, Instance Block, Constructor, Method

Consider:

```
class Employee {  
  
    static int companyId = 100;  
  
    int employeeId = 10;  
  
    static {  
        System.out.println("1. Static block");  
    }  
  
    {  
        System.out.println("2. Instance block");  
    }  
  
    Employee() {  
        System.out.println("3. Constructor");  
    }  
  
    void display() {  
        System.out.println("4. Instance method");  
    }  
}
```

Now:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Employee employee = new Employee();  
  
        employee.display();  
    }  
}
```

Output:

1. Static block

- 2. Instance block
 - 3. Constructor
 - 4. Instance method
-

6. Why Does the Static Block Run First?

Before Java can create the first Employee object, the JVM needs the class to be initialized.

Therefore:

```
static int companyId = 100;
```

and:

```
static {  
    System.out.println("1. Static block");  
}
```

are processed during class initialization.

Only after that can object initialization occur.

7. Static Initialization Algorithm

For a class, conceptually think of the process as:

```
START  
↓  
Load class  
↓  
Allocate static storage  
↓  
Give static fields default values  
↓  
Execute static field initializers  
↓  
Execute static initializer blocks  
↓  
In source-code order  
↓  
Class initialization complete
```

For example:

```
class Test {
```

```
static int a = 10;

static {
    System.out.println(a);
}

static int b = 20;

static {
    System.out.println(b);
}
}
```

Execution:

```
static int a = 10
    ↓
static block
    ↓
static int b = 20
    ↓
static block
```

Output:

```
10
20
```

8. Static Variables and Static Blocks Execute in Source-Code Order

This is extremely important.

Consider:

```
class Test {

    static int a = 10;

    static {
        System.out.println("A = " + a);
    }

    static int b = 20;

    static {
        System.out.println("B = " + b);
    }
}
```

Execution:

a = 10
↓
static block
↓
b = 20
↓
static block

9. Static Method Is Not Automatically Executed

Consider:

```
class Test {  
  
    static int x = 10;  
  
    static {  
        System.out.println("Static block");  
    }  
  
    static void display() {  
        System.out.println("Static method");  
    }  
}
```

If the class is initialized, Java does **not** automatically execute:

display();

The method executes only when explicitly called:

Test.display();

Therefore:

Static variable initialization
↓
Static block
↓
Class initialization complete
↓
Static method executes only if called

10. Instance Initialization Algorithm

When you execute:

```
Employee employee = new Employee();
```

the instance initialization process is approximately:

```
START
↓
Allocate memory for object
↓
Set instance fields to default values
↓
Execute superclass constructor process
↓
Execute subclass instance field initializers
↓
Execute instance initializer blocks
↓
Execute constructor body
↓
Object initialization complete
```

For a class without inheritance, the simplified model is:

```
new Employee()
↓
Default instance values
↓
Instance variable initializers
↓
Instance initializer blocks
↓
Constructor
↓
Object ready
```

11. Instance Variables and Instance Blocks Execute in Source-Code Order

Consider:

```
class Employee {

    int id = 100;

    {
        System.out.println("Block 1: id = " + id);
    }
}
```

```
String name = "Anurag";

{
    System.out.println("Block 2: name = " + name);
}

Employee() {
    System.out.println("Constructor");
}
}
```

Create the object:

```
Employee e = new Employee();
```

Output:

```
Block 1: id = 100
Block 2: name = Anurag
Constructor
```

The order is:

```
id initializer
↓
instance block 1
↓
name initializer
↓
instance block 2
↓
constructor
```

12. Instance Methods Are Different

Consider:

```
class Employee {

    int salary = 50000;

    {
        System.out.println("Instance block");
        display();
    }

    Employee() {
        System.out.println("Constructor");
    }
}
```

```

    }

    void display() {
        System.out.println("Salary = " + salary);
    }
}

```

Creating:

```
Employee e = new Employee();
```

Output:

```

Instance block
Salary = 50000
Constructor

```

Why?

Because the instance initializer explicitly calls:

```
display();
```

The method itself isn't automatically executed merely because it exists.

13. Static Block Can Call Static Methods

Example:

```

class Employee {

    static int salary = 50000;

    static {
        display();
    }

    static void display() {
        System.out.println("Salary = " + salary);
    }
}

```

The sequence is:

```

static salary = 50000
    ↓
static block

```

↓
display()
↓
static method executes

Output:

Salary = 50000

14. Static Block Cannot Directly Access Instance Variables

This is invalid:

```
class Employee {  
    int salary = 50000;  
  
    static {  
        System.out.println(salary); // ERROR  
    }  
}
```

Why?

A static block belongs to the **class**.

At the time it executes, there may not be an Employee object.

There could be:

Employee object 1 → salary = 50000
Employee object 2 → salary = 70000
Employee object 3 → salary = 90000

Which salary should the static block use?

There is no particular object.

Therefore:

```
static {  
    System.out.println(salary);  
}
```

is invalid.

15. Instance Block Can Access Static and Instance Variables

This is valid:

```
class Employee {  
    static int companyId = 100;  
    int employeeId = 200;  
    {  
        System.out.println(companyId);  
        System.out.println(employeeId);  
    }  
}
```

An object has access to:

Class-level data
+
Object-level data

Therefore the instance block can access both.

16. Static Block vs Instance Block

Consider:

```
class Test {  
    static int staticValue = 10;  
    int instanceValue = 20;  
    static {  
        System.out.println("Static: " + staticValue);  
    }  
    {  
        System.out.println("Instance: " + instanceValue);  
    }  
}
```

Now:

```
Test t1 = new Test();  
Test t2 = new Test();
```

Output:

Static: 10
Instance: 20
Instance: 20

Notice:

Static block → 1 time
Instance block → 2 times

because there are two objects.

17. Complete Example

Let's combine variables, blocks, constructors, and methods.

```
class Employee {  
  
    // Static variable  
    static int companyId = 100;  
  
    // Instance variable  
    int employeeId = 200;  
  
    // Static initializer  
    static {  
        System.out.println("1. Static initializer");  
    }  
  
    // Instance initializer  
    {  
        System.out.println("2. Instance initializer");  
    }  
  
    // Constructor  
    Employee() {  
        System.out.println("3. Constructor");  
    }  
  
    // Static method  
    static void staticMethod() {  
        System.out.println("4. Static method");  
    }  
  
    // Instance method  
    void instanceMethod() {
```

```
        System.out.println("5. Instance method");
    }
}
```

Main:

```
public class Main {

    public static void main(String[] args) {

        System.out.println("Main started");

        Employee.staticMethod();

        Employee e1 = new Employee();

        e1.instanceMethod();

        Employee e2 = new Employee();

        e2.instanceMethod();
    }
}
```

Output:

Main started
1. Static initializer
4. Static method
2. Instance initializer
3. Constructor
5. Instance method
2. Instance initializer
3. Constructor
5. Instance method

18. Why Does the Static Block Run Before the Static Method?

The statement:

```
Employee.staticMethod();
```

causes Employee to be actively initialized if it hasn't already been initialized.

Therefore:

Employee class initialization

↓

static variable initialization



static block



staticMethod()

The method itself does not cause static initialization to run repeatedly.

Once the class is initialized:

static block

doesn't execute again merely because another static method is called.

19. Important Point About Static Initialization

Static initialization happens **once per class initialization**, not once per object.

Example:

```
Employee e1 = new Employee();  
Employee e2 = new Employee();  
Employee e3 = new Employee();
```

Conceptually:

Class initialization



Static block



Object 1



Instance block



Constructor



Object 2



Instance block



Constructor



Object 3



Instance block



Constructor

So:

Static block → 1
Instance block → 3
Constructor → 3

20. Multiple Static Blocks

Java allows multiple static blocks.

```
class Test {  
  
    static int a = 10;  
  
    static {  
        System.out.println("Static Block 1");  
    }  
  
    static int b = 20;  
  
    static {  
        System.out.println("Static Block 2");  
    }  
  
    static int c = 30;  
  
    static {  
        System.out.println("Static Block 3");  
    }  
}
```

Execution:

```
a = 10  
↓  
Static Block 1  
↓  
b = 20  
↓  
Static Block 2  
↓  
c = 30  
↓  
Static Block 3
```

21. Multiple Instance Blocks

Similarly:

```
class Test {  
  
    int a = 10;  
  
    {  
        System.out.println("Instance Block 1");  
    }  
  
    int b = 20;  
  
    {  
        System.out.println("Instance Block 2");  
    }  
  
    int c = 30;  
  
    {  
        System.out.println("Instance Block 3");  
    }  
  
    Test() {  
        System.out.println("Constructor");  
    }  
}
```

Execution:

```
a = 10  
↓  
Instance Block 1  
↓  
b = 20  
↓  
Instance Block 2  
↓  
c = 30  
↓  
Instance Block 3  
↓  
Constructor
```

22. The Constructor's Role

An instance initializer does not execute independently from the constructor.

The compiler effectively incorporates instance initializer code into each constructor, after the superclass constructor invocation and before the constructor's own body.

For example:

```
class Employee {  
    int salary = 50000;  
  
    {  
        System.out.println("Instance block");  
    }  
  
    Employee() {  
        System.out.println("Constructor");  
    }  
}
```

Conceptually, you can think of the constructor as behaving approximately like:

```
Employee() {  
  
    // instance field initialization  
    salary = 50000;  
  
    // instance initializer  
    System.out.println("Instance block");  
  
    // original constructor body  
    System.out.println("Constructor");  
}
```

This is a conceptual model rather than the literal source-to-source transformation performed by the compiler.

23. Variable → Block → Constructor → Method

A useful interview model is:

Class level

```
Static variable initialization  
    ↓  
Static initializer block  
    ↓  
Class initialization complete
```

Object level

Instance variable initialization



Instance initializer block



Constructor



Object ready

Method level

Method declaration



No automatic execution



Method executes only when called

24. What Happens with Inheritance?

This is where the sequence becomes particularly important.

Consider:

```
class Parent {  
    static int x = 10;  
  
    static {  
        System.out.println("Parent static block");  
    }  
  
    int a = 20;  
  
    {  
        System.out.println("Parent instance block");  
    }  
  
    Parent() {  
        System.out.println("Parent constructor");  
    }  
}
```

Child:

```
class Child extends Parent {  
    static int y = 30;
```

```

static {
    System.out.println("Child static block");
}

int b = 40;

{
    System.out.println("Child instance block");
}

Child() {
    System.out.println("Child constructor");
}
}

```

Now:

```
Child c = new Child();
```

Output:

```

Parent static block
Child static block
Parent instance block
Parent constructor
Child instance block
Child constructor

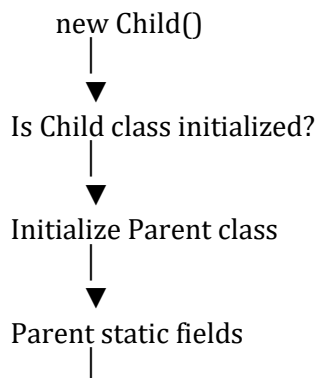
```

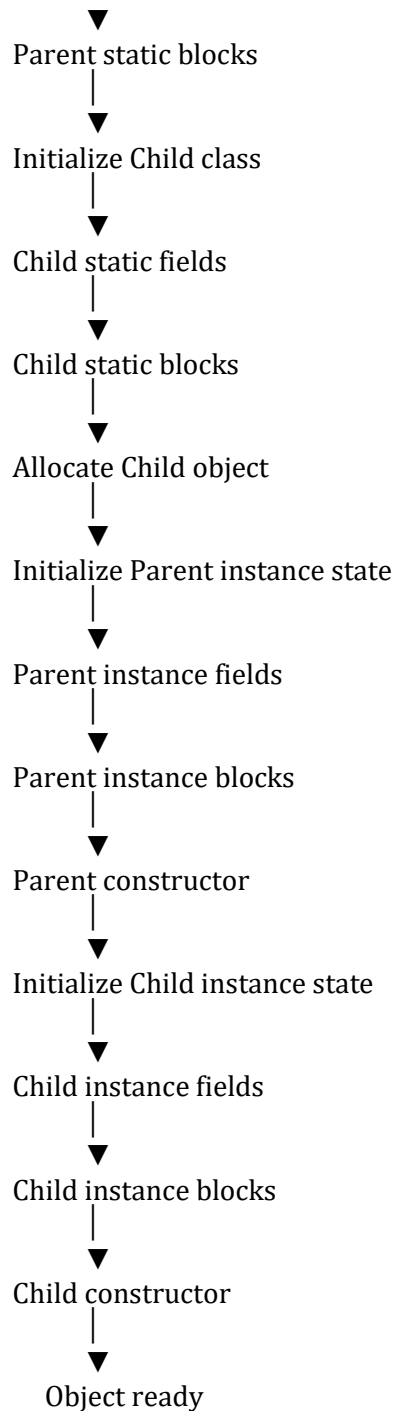
25. Inheritance Initialization Algorithm

For:

```
new Child();
```

the high-level sequence is:

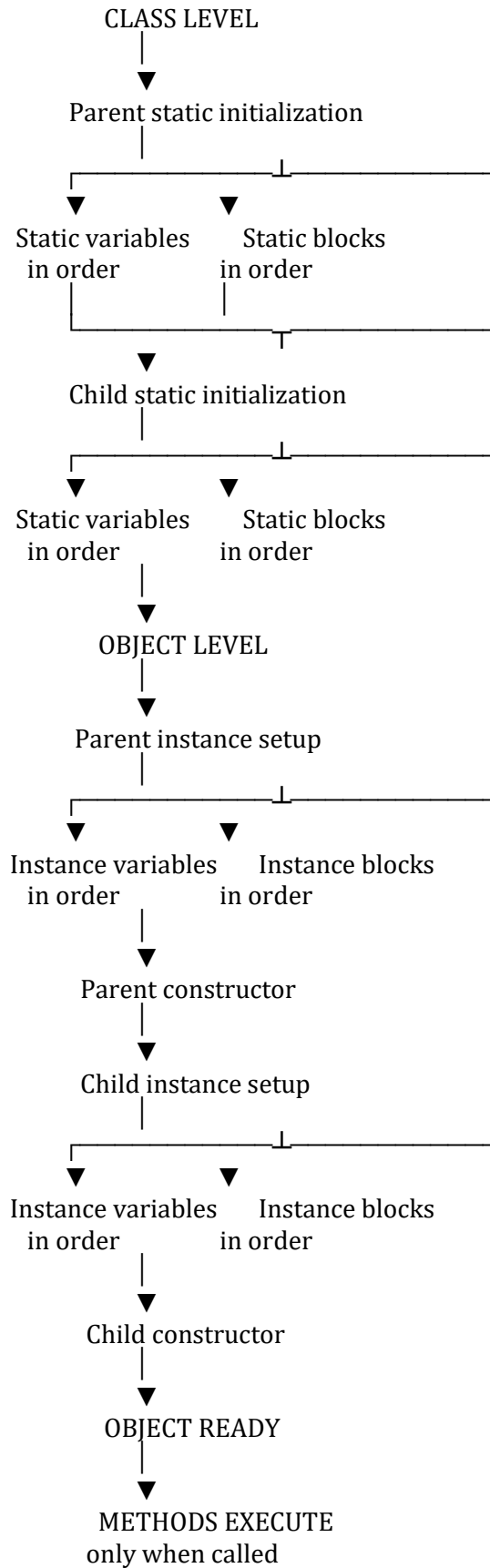




This is one of the most important Java initialization sequences.

26. Full Variable/Block/Method/Constructor Sequence

For a class hierarchy:



27. Static Method vs Instance Method

This is another important distinction.

Static method

```
static void display() {  
    System.out.println("Static");  
}
```

It belongs to the class.

Call:

```
Employee.display();
```

Instance method

```
void display() {  
    System.out.println("Instance");  
}
```

It belongs to an object.

Call:

```
Employee e = new Employee();  
e.display();
```

Neither type of method is automatically executed simply because it is declared.

28. Static Block vs Static Method

Do not confuse:

```
static {  
    System.out.println("Block");  
}
```

with:

```
static void method() {  
    System.out.println("Method");  
}
```

```
}
```

The static block is automatically executed during class initialization.

The static method is executed only when explicitly called.

static block

↓

automatic during class initialization

static method

↓

requires method call

29. Instance Block vs Instance Method

Similarly:

```
{  
    System.out.println("Instance block");  
}
```

executes automatically during object construction.

But:

```
void method() {  
    System.out.println("Instance method");  
}
```

doesn't execute automatically.

instance block

↓

automatic during construction

instance method

↓

requires method call

30. A Very Important Example

Consider:

```
class Test {  
  
    int x = 10;
```

```

{
    System.out.println("Block 1: " + x);
}

int y = 20;

void display() {
    System.out.println("Method: " + y);
}

{
    System.out.println("Block 2: " + y);
    display();
}

Test() {
    System.out.println("Constructor: " + x + ", " + y);
}
}

```

Create:

```
Test t = new Test();
```

Output:

```

Block 1: 10
Block 2: 20
Method: 20
Constructor: 10, 20

```

Why?

Source order:

```

x = 10
↓
Block 1
↓
y = 20
↓
Block 2
↓
display() ← explicitly called
↓
Constructor

```

The method doesn't run because it is a method; it runs because the initializer explicitly calls it.

31. Default Values Matter

Before explicit initialization, Java fields receive default values.

For example:

```
class Test {  
    int x;  
  
    {  
        System.out.println(x);  
    }  
}
```

Output:

0

For an object reference:

String name;

the default is:

null

For a boolean:

false

For reference types:

null

For numeric primitives:

0

Therefore the conceptual sequence is:

Memory allocated

↓

Fields receive default values

↓

Explicit field initializers

↓
Initializer blocks
↓
Constructor

32. Static Fields Also Receive Default Values

Consider:

```
class Test {  
    static int x;  
  
    static {  
        System.out.println(x);  
    }  
}
```

Output:

0

The static field first receives its default value, then the static initializer executes.

33. Static Initialization Happens Before main() Uses the Class

Consider:

```
class Test {  
    static {  
        System.out.println("Static block");  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Main");  
    }  
}
```

Output:

Static block
Main

Why?

The JVM initializes Test before invoking its main method.

34. Important Exception: Compile-Time Constants

There is an important nuance for interviews.

Consider:

```
class Test {  
    static final int X = 100;  
  
    static {  
        System.out.println("Static block");  
    }  
}
```

X is a compile-time constant.

Using:

```
System.out.println(Test.X);
```

does not necessarily trigger class initialization, because the compiler can inline the constant value.

For example:

```
System.out.println(Test.X);
```

may effectively become:

```
System.out.println(100);
```

This is why the statement:

"Any reference to a static field initializes the class"

is **not universally correct**.

The details of JVM class initialization are more precise than that.

35. Initialization Sequence: Interview Version

For a normal class:

CLASS INITIALIZATION

1. Allocate static storage
2. Set static fields to default values
3. Execute static field initializers
4. Execute static initializer blocks
5. Follow source-code order
6. Class initialization completes

Then for:

```
new MyClass();
```

OBJECT INITIALIZATION

1. Allocate object
2. Set instance fields to default values
3. Initialize superclass instance state
4. Execute instance field initializers
5. Execute instance initializer blocks
6. Execute constructor body
7. Object initialization completes

And:

METHODS

Methods are not automatically executed.

They execute only when called.

36. Complete Example for Interview Practice

```
class Parent {  
  
    static int a = 10;  
  
    static {  
        System.out.println("Parent static variable: " + a);  
    }  
  
    int x = 20;  
  
    {  
        System.out.println("Parent instance variable: " + x);  
    }  
}
```

```

Parent() {
    System.out.println("Parent constructor");
}

static void staticMethod() {
    System.out.println("Parent static method");
}

void instanceMethod() {
    System.out.println("Parent instance method");
}
}

class Child extends Parent {

    static int b = 30;

    static {
        System.out.println("Child static variable: " + b);
    }

    int y = 40;

    {
        System.out.println("Child instance variable: " + y);
    }

    Child() {
        System.out.println("Child constructor");
    }

    static void childStaticMethod() {
        System.out.println("Child static method");
    }

    void childInstanceMethod() {
        System.out.println("Child instance method");
    }
}

```

Main:

```

public class Main {

    public static void main(String[] args) {

        System.out.println("MAIN START");

        Child.staticMethod();
    }
}

```

```

    Child c = new Child();

    c.instanceMethod();
    c.childInstanceMethod();
    Child.childStaticMethod();
}
}

```

The important sequence is:

```

MAIN START
↓
Parent class initialization
↓
Parent static initialization
↓
Child class initialization
↓
Child static initialization
↓
Parent instance initialization
↓
Parent constructor
↓
Child instance initialization
↓
Child constructor
↓
Explicit instance method calls
↓
Explicit static method call

```

37. Static vs Instance: Conceptual Model

The easiest way to remember the entire topic is:

Static = Class

```

Class
|
├── static variable
├── static block
└── static method

```

Static variables and blocks participate in **class initialization**.

Static methods execute only when called.

Instance = Object

Object

- instance variable
- instance block
- constructor
- instance method

Instance variables and blocks participate in **object initialization**.

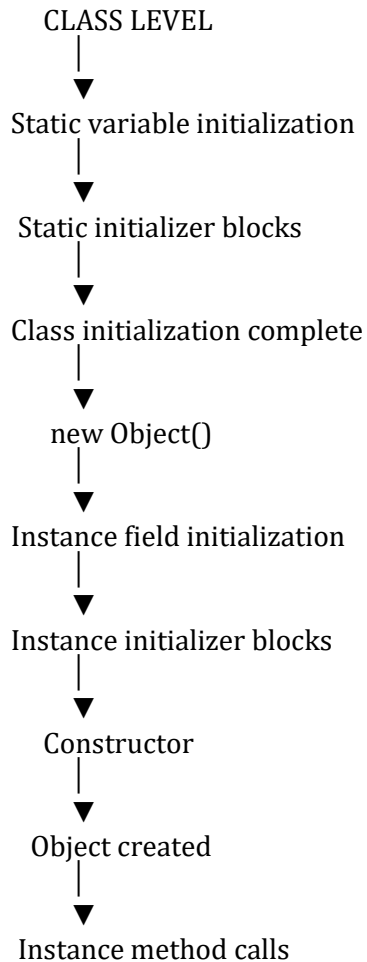
Instance methods execute only when called.

38. Final Execution Matrix

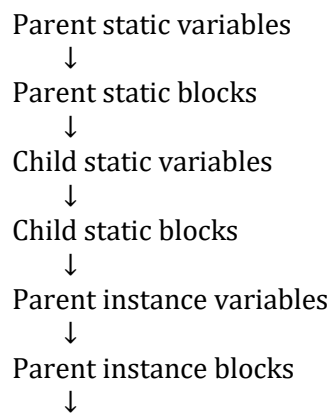
Element	Belongs To	Automatically Executes?	When?	Frequency
Static variable initializer	Class	Yes	Class initialization	Once per class initialization
Static initializer block	Class	Yes	Class initialization	Once per class initialization
Static method	Class	No	When called	Every call
Instance variable initializer	Object	Yes	Object initialization	Once per object
Instance initializer block	Object	Yes	Object initialization	Once per object
Constructor	Object creation	Yes, as part of new	Object initialization	Once per object
Instance method	Object	No	When called	Every call

39. The Most Important Sequence to Memorize

For a **single class**:



For **inheritance**:



Parent constructor
↓
Child instance variables
↓
Child instance blocks
↓
Child constructor
↓
Object ready
↓
Methods execute when explicitly called

One-line interview answer

A static initializer initializes class-level state once when the class is initialized, whereas an instance initializer initializes object-level state every time an object is created. Static variables and static blocks execute during class initialization; instance variables, instance initializer blocks, and constructors execute during object initialization; methods are not automatically executed and run only when explicitly invoked.

Functional Interface, Lambda Expression, and Method Reference in Static and Instance Initializers

This topic combines **three different Java 8 concepts** with **Java class/object initialization order**:

1. **Functional interface** — the target type that describes the function.
2. **Lambda expression** — supplies an implementation of that functional interface.
3. **Method reference** — supplies an existing method as the implementation of that functional interface.
4. **Static initializer / instance initializer** — determines **when the functional object is created and when the referenced behavior actually executes**.

The most important rule is:

Creating a lambda or method reference does not normally execute its body or referenced method. Invocation of the functional interface is what executes the behavior.

For example:

```
Function<Integer, Integer> f = x -> square(x);
```

creates a function.

It does **not** normally execute `square()` at that point.

Likewise:

```
Function<Integer, Integer> f = MyClass::square;
```

creates a function referring to `square()`.

It does **not** call `square()` at that point.

1. The Three Concepts

1.1 Functional Interface

A **functional interface** is an interface containing exactly one abstract method.

Example:

```
@FunctionalInterface
interface Calculator {
    int calculate(int x);
}
```

The abstract method is:

```
int calculate(int x);
```

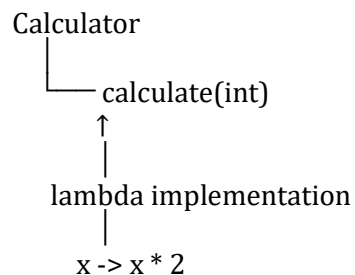
A lambda or method reference can provide its implementation.

1.2 Lambda Expression

A lambda expression provides behavior for a functional interface.

```
Calculator c = x -> x * 2;
```

Conceptually:



When:

```
c.calculate(10);
```

is executed, the lambda body executes:

```
10 * 2  
↓  
20
```

2. Method Reference

A method reference refers to an existing method.

Example:

```
class CalculatorUtil {  
    static int doubleValue(int x) {  
        return x * 2;  
    }  
}
```

Method reference:

```
Calculator c = CalculatorUtil::doubleValue;
```

Equivalent lambda:

```
Calculator c = x -> CalculatorUtil.doubleValue(x);
```

Again:

```
Calculator c = CalculatorUtil::doubleValue;
```

does **not** mean:

```
CalculatorUtil.doubleValue(...);
```

is executed immediately.

The method executes when:

```
c.calculate(10);
```

is invoked.

3. The Critical Distinction: Creation vs Invocation

This is the foundation of the entire topic.

Consider:

```
Function<Integer, Integer> function =  
    x -> x * 2;
```

There are two conceptual events:

Event 1 — Lambda creation

Function object/implementation created

Event 2 — Lambda invocation

```
function.apply(10);
```

Then:

```
10  
↓  
lambda body  
↓  
20
```

Similarly:

```
Function<Integer, Integer> function =  
    CalculatorUtil::doubleValue;
```

has:

Event 1

Method-reference function created

Event 2

```
function.apply(10);
```

Then:

```
CalculatorUtil.doubleValue(10)
```

executes.

4. Static Initializer

A static initializer is:

```
static {  
    // code  
}
```

It executes during **class initialization**.

Example:

```
class Demo {  
  
    static {  
        System.out.println("Static initializer");  
    }  
}
```

It normally executes once when the class is initialized.

5. Instance Initializer

An instance initializer is:

```
{  
    // code  
}
```

It executes during construction of each object.

Example:

```
class Demo {  
  
    {  
        System.out.println("Instance initializer");  
    }  
}
```

If:

```
new Demo();
```

is executed twice, the instance initializer executes twice.

6. First Major Algorithm

For static initialization:

Class initialization



Static variable initializers



Static initializer blocks



Class initialization complete

For object initialization:

Object creation



Instance field initializers



Instance initializer blocks



Constructor



Object initialization complete

Methods are different:

Method declaration



Nothing executes automatically



Method executes only when called

7. Functional Interface at the Static Variable Level

Consider:

```
class Employee {  
  
    static Function<Integer, Integer> salaryCalculator =  
        salary -> salary + 1000;  
}
```

When Employee is initialized:

Employee class initialization



salaryCalculator variable initializer



lambda expression evaluated



Function implementation created



salaryCalculator assigned

But:

salary -> salary + 1000

has **not calculated a salary yet**.

There is no input.

The lambda executes only when:

Employee.salaryCalculator.apply(50000);

is called.

8. Static Variable + Lambda + Invocation

```
class Employee {  
  
    static Function<Integer, Integer> salaryCalculator =  
        salary -> {  
            System.out.println("Lambda executing");  
            return salary + 1000;  
        };  
}
```

Now:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println("Before");  
  
        int result =  
            Employee.salaryCalculator.apply(50000);  
  
        System.out.println("Result = " + result);  
    }  
}
```

```
}  
}
```

Output:

Before
Lambda executing
Result = 51000

The important sequence is:

Employee class initialization
↓
static variable initializer
↓
lambda object/function created
↓
class initialization complete
↓
salaryCalculator.apply(50000)
↓
lambda body executes

9. Static Variable + Method Reference

```
class Employee {  
  
    static int calculateSalary(int salary) {  
        System.out.println("Method executing");  
        return salary + 1000;  
    }  
  
    static Function<Integer, Integer> salaryCalculator =  
        Employee::calculateSalary;  
}
```

During class initialization:

salaryCalculator = Employee::calculateSalary

creates the functional implementation.

It does **not** execute:

calculateSalary()

Then:

```
Employee.salaryCalculator.apply(50000);
```

causes:

```
apply(50000)
```

↓

```
Employee.calculateSalary(50000)
```

↓

```
51000
```

10. Static Method Reference vs Lambda

These are approximately equivalent:

Lambda

```
static Function<Integer, Integer> f =  
    x -> calculateSalary(x);
```

Method reference

```
static Function<Integer, Integer> f =  
    Employee::calculateSalary;
```

Both create a function.

Neither normally executes calculateSalary() during the assignment itself.

11. Static Initializer Block

Now put the invocation inside the static block:

```
class Employee {  
  
    static Function<Integer, Integer> calculator =  
        salary -> salary + 1000;  
  
    static {  
        System.out.println("Static block");  
  
        int result = calculator.apply(50000);  
  
        System.out.println(result);  
    }  
}
```

Output:

Static block
51000

Here the lambda **does execute during static initialization**, but not because it is a lambda.

It executes because the static block explicitly invokes:

```
calculator.apply(50000);
```

This distinction is extremely important.

12. Static Initializer with Method Reference

```
class Employee {  
  
    static int calculate(int salary) {  
        System.out.println("calculate() executing");  
        return salary + 1000;  
    }  
  
    static Function<Integer, Integer> calculator =  
        Employee::calculate;  
  
    static {  
        System.out.println("Static block");  
  
        int result = calculator.apply(50000);  
  
        System.out.println("Result = " + result);  
    }  
}
```

Sequence:

1. calculator variable initializer
↓
2. Employee::calculate evaluated
↓
3. Functional object/reference created
↓
4. static block begins
↓
5. calculator.apply(50000)
↓
6. Employee.calculate(50000)

↓
7. Result returned

13. Static Block Can Invoke a Lambda

```
class Demo {  
  
    static Consumer<String> printer =  
        text -> System.out.println(text);  
  
    static {  
        printer.accept("Hello from static block");  
    }  
}
```

Output:

Hello from static block

Algorithm:

```
static printer initialized  
↓  
lambda created  
↓  
static block  
↓  
printer.accept(...)  
↓  
lambda executes
```

14. Static Block Can Invoke a Method Reference

```
class Demo {  
  
    static void print(String text) {  
        System.out.println(text);  
    }  
  
    static Consumer<String> printer =  
        Demo::print;  
  
    static {  
        printer.accept("Hello");  
    }  
}
```

Equivalent lambda:

```
static Consumer<String> printer =  
    text -> Demo.print(text);
```

15. Static Method Level

A static method itself is **not automatically executed** during class initialization.

Example:

```
class Demo {  
  
    static Function<Integer, Integer> calculator =  
        x -> x * 2;  
  
    static void test() {  
        System.out.println("Static method");  
  
        int result = calculator.apply(10);  
  
        System.out.println(result);  
    }  
}
```

Simply initializing the class does not mean:

```
test();
```

runs.

Only:

```
Demo.test();
```

executes it.

Then:

```
Demo.test()  
↓  
calculator.apply(10)  
↓  
lambda body  
↓  
20
```

16. Instance Variable + Lambda

Now consider an instance initializer.

```
class Employee {  
    int bonus = 1000;  
  
    Function<Integer, Integer> calculator =  
        salary -> salary + bonus;  
}
```

When an object is created:

```
Employee e = new Employee();
```

the instance variable initializer executes.

The lambda captures the object's instance state through bonus.

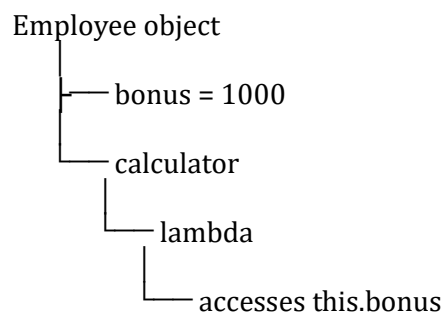
Then:

```
e.calculator.apply(50000);
```

produces:

51000

Conceptually:



17. Why Can the Lambda Access bonus?

Inside an instance context:

```
salary -> salary + bonus
```

is conceptually associated with:

salary -> salary + this.bonus

The lambda can capture the enclosing instance.

Therefore:

Employee e = new Employee();

creates an object whose lambda can access that object's state.

18. Instance Initializer + Lambda

```
class Employee {  
  
    int bonus = 1000;  
  
    Function<Integer, Integer> calculator =  
        salary -> salary + bonus;  
  
    {  
        System.out.println("Instance initializer");  
  
        int result =  
            calculator.apply(50000);  
  
        System.out.println("Result = " + result);  
    }  
}
```

Creating:

Employee e = new Employee();

produces:

Instance initializer
Result = 51000

Sequence:

```
new Employee()  
↓  
instance field bonus = 1000  
↓  
calculator lambda initialized  
↓  
instance initializer block
```

↓
calculator.apply(50000)
↓
lambda executes
↓
salary + bonus
↓
51000

19. Instance Method Reference Using this

Now consider:

```
class Employee {  
  
    int bonus = 1000;  
  
    int calculate(int salary) {  
        return salary + bonus;  
    }  
  
    Function<Integer, Integer> calculator =  
        this::calculate;  
}
```

This is an **instance method reference to the current object**.

this::calculate

means approximately:

salary -> this.calculate(salary)

The method isn't called during the reference creation.

20. Invocation of this::calculate

```
Employee e = new Employee();
```

```
int result =  
    e.calculator.apply(50000);
```

Execution:

```
e.calculator.apply(50000)  
↓
```

```
this.calculate(50000)
    ↓
salary + bonus
    ↓
51000
```

21. Instance Initializer with this::method

```
class Employee {

    int bonus = 1000;

    int calculate(int salary) {
        System.out.println("calculate() executing");
        return salary + bonus;
    }

    Function<Integer, Integer> calculator =
        this::calculate;

    {
        System.out.println("Instance block");

        int result = calculator.apply(50000);

        System.out.println("Result = " + result);
    }
}
```

Creating the object:

```
Employee e = new Employee();
```

Output:

```
Instance block
calculate() executing
Result = 51000
```

22. Important Difference: Employee::calculate vs this::calculate

This is an important Java 8 interview concept.

Suppose:

```
class Employee {
```

```
int calculate(int salary) {  
    return salary + 1000;  
}  
}
```

Arbitrary Employee object

```
Function<Employee, Integer> f =  
    Employee::calculate;
```

This means:

employee -> employee.calculate(...)

The target object is supplied later.

Particular current object

```
Function<Integer, Integer> f =  
    this::calculate;
```

This means:

salary -> this.calculate(salary)

The current object is captured.

23. Static Initializer Cannot Use this

This is invalid:

```
class Employee {  
  
    static Function<Integer, Integer> calculator =  
        this::calculate;  
}
```

Compilation error.

Why?

this refers to a particular object.

A static context does not have a current object.

Therefore:

static context

↓

no this

↓

cannot use this::instanceMethod

24. But Static Context Can Use ClassName::instanceMethod

This is valid:

```
class Employee {  
  
    int calculate(int salary) {  
        return salary + 1000;  
    }  
  
    static Function<Employee, Integer> calculator =  
        Employee::calculate;  
}
```

Why?

Because Employee::calculate represents an instance method whose receiver will be supplied later.

Conceptually:

employee -> employee.calculate(...)

25. Static Context Can Use Static Method Reference

Obviously:

```
class Employee {  
  
    static int calculate(int salary) {  
        return salary + 1000;  
    }  
  
    static Function<Integer, Integer> calculator =  
        Employee::calculate;  
}
```

This is straightforward:

Function

↓

Employee::calculate

↓
Employee.calculate(value)

26. Instance Initializer Can Use Both

Inside an instance context, you can have:

Static method reference

Employee::calculateStatic

Instance method reference to current object

this::calculate

Arbitrary instance method reference

Employee::calculate

depending on the target functional interface signature.

27. Complete Example

Let's combine everything.

```
import java.util.function.Function;

class Employee {

    static int companyBonus = 500;

    int employeeBonus = 1000;

    // Static variable + lambda
    static Function<Integer, Integer> staticLambda =
        salary -> salary + companyBonus;

    // Static variable + static method reference
    static Function<Integer, Integer> staticMethodReference =
        Employee::calculateStatic;

    // Static variable + arbitrary-object instance method reference
    static Function<Employee, Integer> arbitraryMethodReference =
        Employee::calculateInstance;

    static {
```

```

        System.out.println("1. Static block");

        System.out.println(
            staticLambda.apply(50000));

        System.out.println(
            staticMethodReference.apply(50000));
    }

    static int calculateStatic(int salary) {
        return salary + companyBonus;
    }

    int calculateInstance(int salary) {
        return salary + employeeBonus;
    }

    // Instance variable + lambda
    Function<Integer, Integer> instanceLambda =
        salary -> salary + employeeBonus;

    // Instance method reference to this object
    Function<Integer, Integer> instanceMethodReference =
        this::calculateInstance;

    {
        System.out.println("2. Instance block");

        System.out.println(
            instanceLambda.apply(50000));

        System.out.println(
            instanceMethodReference.apply(50000));
    }
}

```

Create the object:

```

public class Main {

    public static void main(String[] args) {

        System.out.println("3. Main");

        Employee employee = new Employee();
    }
}

```

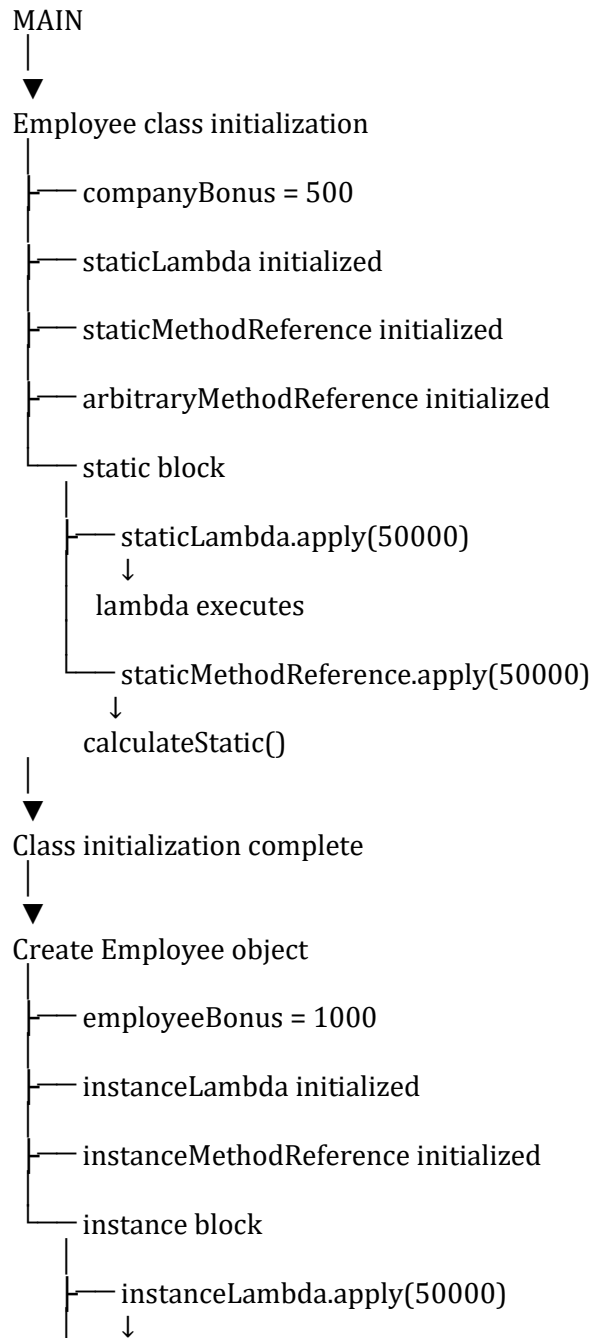
28. Execution Sequence of the Complete Example

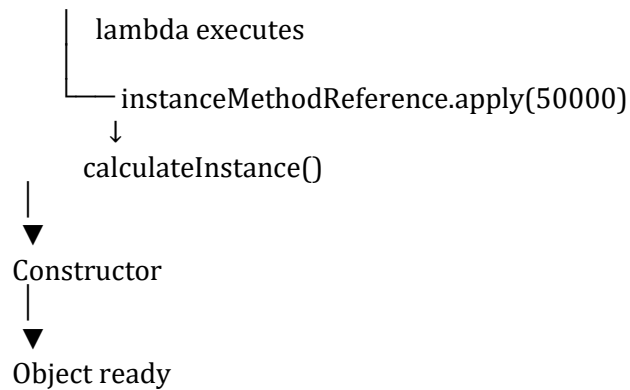
When:

`new Employee();`

is reached, the JVM must first ensure that `Employee` is initialized.

Conceptually:





29. Variable-Level, Block-Level, and Method-Level Algorithm

This distinction is especially useful for your question.

A. Variable Level

Example:

```
Function<Integer, Integer> f = x -> x * 2;
```

Algorithm:

1. Determine target type
↓
2. Determine functional interface
↓
3. Evaluate lambda expression
↓
4. Create/obtain functional implementation
↓
5. Assign it to variable f
↓
6. Lambda body does NOT execute yet

For:

```
Function<Integer, Integer> f =  
    MyClass::calculate;
```

algorithm:

1. Determine target functional interface
↓
2. Resolve referenced method
↓
3. Create/obtain method-reference implementation

- ↓
4. Assign to f
↓
5. Referenced method does NOT execute yet
-

30. Block Level

If the functional interface is invoked inside a block:

```
static {  
    f.apply(10);  
}
```

or:

```
{  
    f.apply(10);  
}
```

then:

Initializer block begins

↓
f.apply(10)
↓
functional interface invocation
↓
lambda/method-reference target executes
↓
result returned
↓
initializer block continues

31. Method Level

If the invocation occurs inside a method:

```
void calculateSomething() {  
    f.apply(10);  
}
```

then nothing happens merely because the method exists.

The sequence is:

Method declaration
↓

No execution
↓
Someone calls method
↓
Method body begins
↓
f.apply(10)
↓
Lambda/method reference executes

32. Very Important: Declaration Does Not Mean Invocation

Compare:

```
Function<Integer, Integer> f =  
    x -> x * 2;
```

with:

```
f.apply(10);
```

The first is **function creation/assignment**.

The second is **function invocation**.

Likewise:

```
Function<Integer, Integer> f =  
    MyClass::calculate;
```

is function/reference creation.

While:

```
f.apply(10);
```

is invocation.

33. Static vs Instance Initialization

Location	Lambda / Method Reference Creation	Can Execute During Initialization?
Static field initializer	Yes	Only if its initializer itself invokes something

Static initializer block	Yes	Yes, if explicitly invoked
Static method	Yes	Yes, when method is called
Instance field initializer	Yes	Yes, as part of object initialization
Instance initializer block	Yes	Yes, if explicitly invoked
Instance method	Yes	Yes, when method is called
Constructor	Yes	Yes, if explicitly invoked

34. The Most Important this Rule

Static initializer

```
static {
    // no this
}
```

Cannot:

```
this::method
```

Instance initializer

```
{
    // this is available
}
```

Can:

```
this::method
```

This makes instance method references particularly interesting in instance field initializers and instance initializer blocks.

35. Constructor Interaction

Consider:

```
class Employee {
```

```

Function<Integer, Integer> calculator =
    this::calculate;

{
    System.out.println(
        calculator.apply(100));
}

Employee() {
    System.out.println("Constructor");
}

int calculate(int x) {
    return x * 2;
}
}

```

Creating:

```
new Employee();
```

causes:

```

Object memory allocated
↓
Instance field initialization
↓
calculator = this::calculate
↓
Instance initializer
↓
calculator.apply(100)
↓
this.calculate(100)
↓
200
↓
Constructor

```

This is legal, but it demonstrates an important design consideration:

Calling overridable instance methods during object initialization can be dangerous, especially in superclass constructors/initializers, because subclass state may not yet be initialized.

36. Inheritance Makes the Sequence More Interesting

Consider:

```

class Parent {

    Function<Integer, Integer> f =
        this::calculate;

    {
        System.out.println(f.apply(10));
    }

    int calculate(int x) {
        return x * 2;
    }
}

```

and:

```

class Child extends Parent {

    int multiplier = 10;

    @Override
    int calculate(int x) {
        return x * multiplier;
    }
}

```

Then:

```
Child c = new Child();
```

can call the overridden Child.calculate() from the parent's initialization **before Child's instance field initializer has assigned multiplier = 10.**

That means multiplier can still contain its default value 0.

This is one reason calling overridable methods from constructors or instance initializers is generally discouraged.

37. Lambda Capture vs Method Reference Capture

This is another useful distinction.

Lambda

```
int bonus = 1000;
```

```
Function<Integer, Integer> f =
```

```
salary -> salary + bonus;
```

The lambda captures the value/state it needs according to Java's capture rules.

For local variables, the captured local variable must be **final or effectively final**.

Instance method reference

```
Function<Integer, Integer> f =  
    this::calculate;
```

The method reference is tied to the current object (this).

Conceptually:

```
method reference  
  ↓  
current Employee object  
  ↓  
calculate()
```

38. ClassName::instanceMethod Is Different

Consider:

```
Function<Employee, Integer> f =  
    Employee::calculateInstance;
```

This does not capture a particular employee.

Instead:

```
f.apply(employee);
```

causes:

```
employee  
  ↓  
employee.calculateInstance(...)
```

Conceptually:

```
employee -> employee.calculateInstance(...)
```

This is different from:

```
Function<Integer, Integer> f =
```

```
this::calculateInstance;
```

which is conceptually:

```
salary -> this.calculateInstance(salary)
```

39. Final Algorithm

Here is the complete algorithm you can use for interview questions.

Step 1 — Class initialization

When the class must be initialized:

Static fields
↓
Static initializer blocks
↓
in source-code order

If a field contains:

lambda

or:

method reference

the functional implementation is created/resolved.

The lambda body or referenced method does not execute merely because the reference is created.

Step 2 — Static initializer invocation

If the static block contains:

```
f.apply(value);
```

then:

static block
↓
functional interface invocation
↓

lambda body / referenced method

↓

result

Step 3 — Object creation

When:

```
new Employee();
```

occurs:

Object allocated

↓

Superclass initialization

↓

Instance fields

↓

Instance initializer blocks

↓

Constructor

Step 4 — Instance lambda/method reference

When an instance field contains:

```
f = x -> ...
```

or:

```
f = this::method;
```

the functional implementation is initialized during object initialization.

Again:

reference creation

≠

method execution

Step 5 — Instance initializer invocation

If:

```
{
```

```
f.apply(10);  
}
```

exists:

instance initializer



f.apply(10)



lambda/method reference invoked



target behavior executes

Step 6 — Constructor

After instance field initializers and instance initializer blocks:

Constructor body

executes.

Step 7 — Ordinary methods

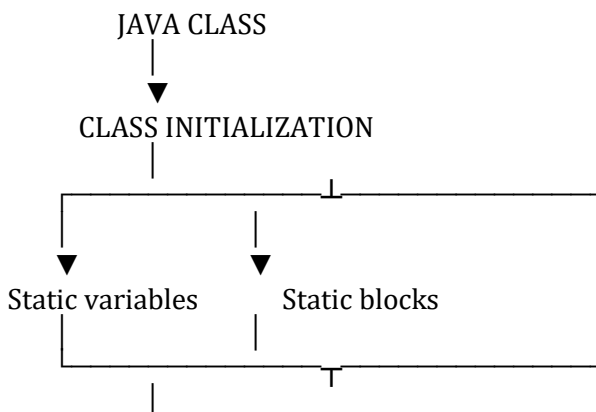
Methods execute only when explicitly invoked:

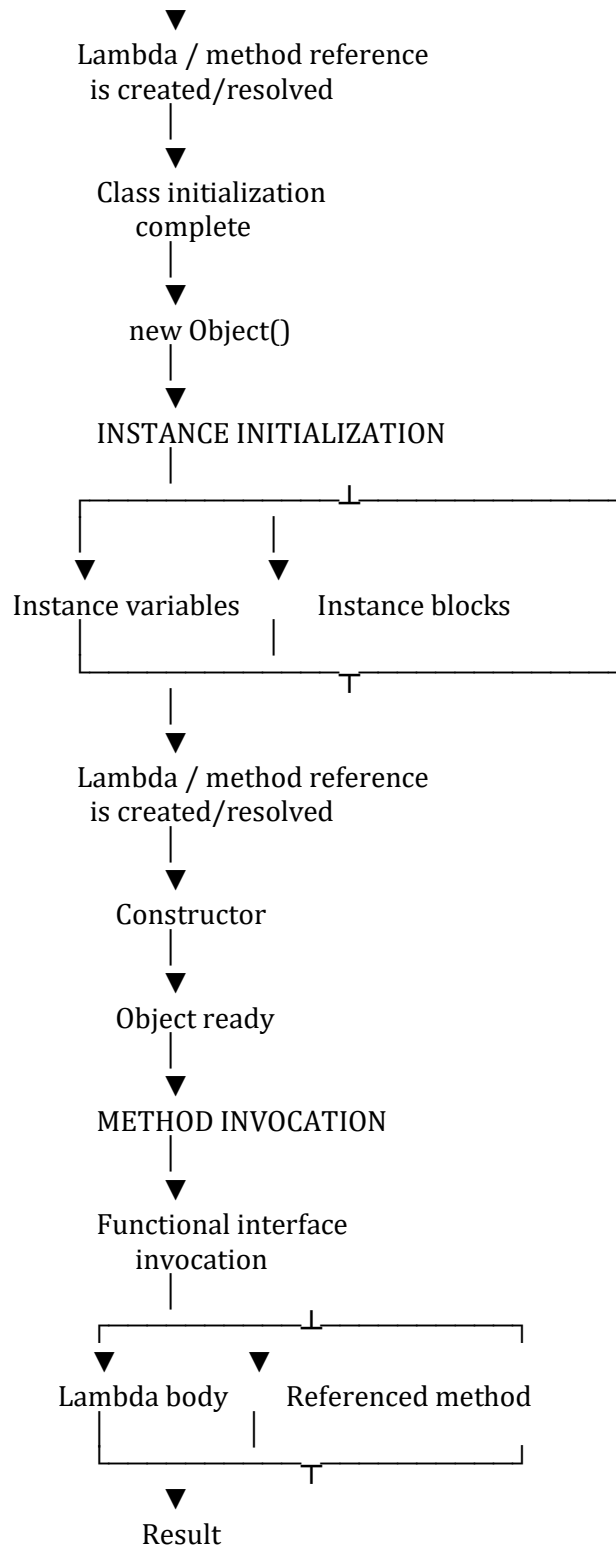
employee.calculate();

or:

function.apply(...);

40. Master Execution Diagram





41. Final Comparison

Concept	What it represents	When	When body executes
---------	--------------------	------	--------------------

		created/evaluated	
Functional interface variable	Target type/reference to functional behavior	During variable initialization	When abstract method is invoked
Lambda expression	Implementation of functional interface	When lambda expression is evaluated	When functional interface method is invoked
Static method reference	Reference to static method	When expression is evaluated	When functional interface method is invoked
Instance method reference this::method	Reference bound to current object	During instance initialization	When functional interface method is invoked
Arbitrary instance method reference Class::method	Function whose receiver is supplied later	When expression is evaluated	When functional interface method is invoked
Constructor reference Class::new	Object-creation behavior	When expression is evaluated	When functional interface method is invoked

42. The Key Rules to Memorize

Rule 1

```
Function<Integer, Integer> f = x -> x * 2;
```

Creates a function; does not execute $x * 2$.

Rule 2

```
Function<Integer, Integer> f = MyClass::calculate;
```

Creates/resolves a method reference; does not call calculate().

Rule 3

```
f.apply(10);
```

Invokes the functional interface and therefore executes the lambda or referenced method.

Rule 4

```
static {  
    f.apply(10);  
}
```

means the functional behavior executes **during class initialization**.

Rule 5

```
{  
    f.apply(10);  
}
```

means the functional behavior executes **during object initialization**.

Rule 6

this::calculate

is possible in an **instance context**, because this represents the current object.

Rule 7

this::calculate

is **not possible in a static context**, because static code has no this.

Rule 8

Employee::calculate

can refer to an instance method even from a static context, provided the functional interface's signature supplies the Employee receiver.

43. One-Line Interview Answer

A functional interface provides the target type, while a lambda expression or method reference provides its implementation. During static or instance variable initialization, evaluating a lambda or method reference normally creates/resolves the functional behavior but does not execute its body. The lambda body or referenced method executes only when the functional interface's abstract method is invoked. Therefore, if apply(), accept(), test(), etc. is called inside a static initializer, the behavior executes during class initialization; if called inside an instance initializer, it executes during object initialization; and if called inside an ordinary method, it executes when that method is invoked.